

Tutorial: API REST de Clientes

com Flask e SQLite

Back-end RESTful com módulos separados de banco de dados e API

Versao 1.0 | 2026

Introdução

Este tutorial orienta a criação de uma API REST para o cadastro de clientes utilizando Flask como framework web e SQLite como banco de dados. A aplicação reutiliza integralmente o módulo `database.py` apresentado no tutorial anterior e adiciona um novo módulo `api.py` dedicado exclusivamente as rotas HTTP.

Ao final, você terá uma API capaz de:

- Gerenciar cidades: inclusão, listagem e exclusão
- Gerenciar clientes: CRUD completo (criar, listar, buscar por ID, atualizar e excluir)
- Retornar respostas em formato JSON com códigos HTTP semânticos
- Tratar erros de forma consistente

Estrutura de módulos:

- `database.py` — Operações com SQLite (reutilizado do tutorial anterior)
- `api.py` — Rotas Flask e lógica de apresentação da API

Passo 1: Preparação do Ambiente

1.1 Pré-requisitos

Certifique-se de ter o Python 3.8 ou superior instalado. O módulo `database.py` do tutorial anterior deve estar presente na mesma pasta do projeto.

1.2 Criando o Ambiente Virtual

No Windows:

```
python -m venv venv
venv\Scripts\activate
```

No Linux / macOS:

```
python -m venv venv
source venv/bin/activate
```

1.3 Instalando as Dependências

Com o ambiente ativado, instale o Flask:

```
pip install flask
```

O módulo `sqlite3` já faz parte da biblioteca padrão do Python e não precisa ser instalado. Para verificar o Flask:

```
flask --version
```

1.4 Estrutura do Projeto

```
cadastro_clientes/
  database.py    # módulo de banco de dados (tutorial anterior)
  api.py        # Novo módulo: rotas Flask
```

Passo 2: Revisão do módulo database.py

O módulo database.py é utilizado sem alterações. Ele expõe as seguintes funções que serão consumidas pela API:

Função	Descrição
criar_tabelas()	Cria as tabelas cidades e clientes caso não existam
listar_cidades()	Retorna lista de dicionários com id e nome de cada cidade
incluir_cidade(nome)	Insere uma cidade; retorna True em caso de sucesso
excluir_cidade(cidade_id)	Remove cidade se não houver clientes vinculados
listar_clientes()	Retorna todos os clientes com JOIN na tabela cidades
buscar_cliente(cliente_id)	Retorna dicionário do cliente ou None se inexistente
incluir_cliente(nome, email, cidade_id)	Insere novo cliente; False se e-mail duplicado
alterar_cliente(id, nome, email, cidade_id)	Atualiza dados do cliente
excluir_cliente(cliente_id)	Remove o cliente pelo ID

Passo 3: módulo da API (api.py)

3.1 Configuração Inicial

O arquivo api.py inicializa o Flask, importa o módulo de banco de dados e garante que as tabelas existam antes de receber requisições:

```
# api.py
from flask import Flask, jsonify, request
import database as db

app = Flask(__name__)

# Garante que as tabelas existam ao iniciar
db.criar_tabelas()
```

3.2 Convenções Adotadas

A API segue as convenções REST abaixo:

- Todas as respostas utilizam Content-Type: application/json
- Códigos HTTP semânticos: 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 409 Conflict, 500 Internal Server Error
- {"erro": "mensagem"} para erros e {"mensagem": "texto"} para confirmações
- IDs são sempre inteiros positivos passados na URL
- Campos obrigatórios ausentes ou vazios retornam 400

3.3 Visão Geral dos Endpoints

Método	Rota	Descrição
GET	/cidades	Lista todas as cidades
POST	/cidades	Cadastra uma nova cidade
DELETE	/cidades/<id>	Remove uma cidade pelo ID
GET	/clientes	Lista todos os clientes
GET	/clientes/<id>	Busca um cliente pelo ID
POST	/clientes	Cadastra um novo cliente
PUT	/clientes/<id>	Atualiza os dados de um cliente
DELETE	/clientes/<id>	Remove um cliente pelo ID

Passo 4: Endpoints de Cidades

GET /cidades *Lista todas as cidades*

Retorna um array JSON com todas as cidades cadastradas, ordenadas por nome. Não exige parâmetros.

```
@app.route("/cidades", methods=["GET"])
def get_cidades():
    cidades = db.listar_cidades()
    return jsonify(cidades), 200
```

Exemplo de resposta (200 OK):

```
[
  { "id": 1, "nome": "Belo Horizonte" },
  { "id": 2, "nome": "Sao Paulo" }
]
```

POST /cidades *Cadastra uma nova cidade*

Recebe um objeto JSON com o campo nome e insere a cidade no banco. Retorna 409 se o nome já existir.

```
@app.route("/cidades", methods=["POST"])
def post_cidade():
    dados = request.get_json(silent=True) or {}
    nome = dados.get("nome", "").strip()
    if not nome:
        return jsonify({"erro": "O campo 'nome' e obrigatorio."}), 400
    if db.incluir_cidade(nome):
        return jsonify({"mensagem": f"Cidade '{nome}' cadastrada com sucesso."}), 201
    return jsonify({"erro": "Ja existe uma cidade com esse nome."}), 409
```

Corpo da requisição (JSON):

nome *	string – nome da cidade (obrigatorio)
---------------	---------------------------------------

Respostas possíveis:

201 Created	{ "mensagem": "Cidade 'Curitiba' cadastrada com sucesso." }
400 Bad Request	{ "erro": "O campo 'nome' e obrigatorio." }
409 Conflict	{ "erro": "Ja existe uma cidade com esse nome." }

DELETE /cidades/<id> *Remove uma cidade pelo ID*

Remove a cidade identificada pelo parâmetro de rota id. A exclusão é bloqueada se houver clientes vinculados a essa cidade.

```
@app.route("/cidades/<int:cidade_id>", methods=["DELETE"])
def delete_cidade(cidade_id):
    resultado = db.excluir_cidade(cidade_id)
```

```

    if resultado is None:
        return jsonify({"erro": "Cidade não encontrada."}), 404
    if not resultado:
        return jsonify({
            "erro": "Não é possível excluir: há clientes vinculados a esta
cidade."
        }), 409
    return jsonify({"mensagem": "Cidade removida com sucesso."}), 200

```

Nota: a função `excluir_cidade` do `database.py` retorna `False` quando há clientes vinculados. Para diferenciar 'não encontrada' de 'com vínculo', você pode estender a função para retornar `None` quando o ID não existir, conforme o trecho acima.

Respostas possíveis:

200 OK	{ "mensagem": "Cidade removida com sucesso." }
404 Not Found	{ "erro": "Cidade não encontrada." }
409 Conflict	{ "erro": "Não é possível excluir: há clientes vinculados a esta cidade." }

Passo 5: Endpoints de Clientes

GET `/clientes` *Lista todos os clientes*

Retorna um array JSON com todos os clientes, incluindo o nome da cidade resolvido via JOIN. Não exige parâmetros.

```
@app.route("/clientes", methods=["GET"])
def get_clientes():
    clientes = db.listar_clientes()
    return jsonify(clientes), 200
```

Exemplo de resposta (200 OK):

```
[
  {
    "id": 1,
    "nome": "Ana Lima",
    "email": "ana@email.com",
    "cidade": "Belo Horizonte",
    "cidade_id": 1
  }
]
```

GET `/clientes/<id>` *Busca um cliente pelo ID*

Retorna os dados de um único cliente identificado pelo parâmetro de rota id. Retorna 404 se o cliente não existir.

```
@app.route("/clientes/<int:cliente_id>", methods=["GET"])
def get_cliente(cliente_id):
    cliente = db.buscar_cliente(cliente_id)
    if cliente is None:
        return jsonify({"erro": "Cliente nao encontrado."}), 404
    return jsonify(cliente), 200
```

Exemplo de resposta (200 OK):

```
{
  "id": 1,
  "nome": "Ana Lima",
  "email": "ana@email.com",
  "cidade_id": 1
}
```

Respostas possíveis:

200 OK	Dados do cliente em JSON
404 Not Found	{ "erro": "Cliente nao encontrado." }

POST `/clientes` *Cadastra um novo cliente*

Recebe um objeto JSON com os campos nome, e-mail e cidade_id. Retorna 409 se o e-mail já estiver cadastrado.


```
@app.route("/clientes", methods=["POST"])
def post_cliente():
    dados = request.get_json(silent=True) or {}
    nome = dados.get("nome", "").strip()
    email = dados.get("email", "").strip()
    cidade_id = dados.get("cidade_id")

    if not nome or not email or not cidade_id:
        return jsonify({
            "erro": "Os campos 'nome', 'email' e 'cidade_id' são obrigatórios."
        }), 400

    if db.incluir_cliente(nome, email, int(cidade_id)):
        return jsonify({"mensagem": "Cliente cadastrado com sucesso."}), 201
    return jsonify({"erro": "E-mail já cadastrado para outro cliente."}), 409
```

Corpo da requisição (JSON):

nome *	string – nome completo do cliente
email *	string – e-mail único do cliente
cidade_id *	integer – ID de uma cidade existente

Respostas possíveis:

201 Created	{ "mensagem": "Cliente cadastrado com sucesso." }
400 Bad Request	{ "erro": "Os campos nome, e-mail e cidade_id são obrigatórios." }
409 Conflict	{ "erro": "E-mail já cadastrado para outro cliente." }

PUT /clientes/<id> *Atualiza os dados de um cliente*

Recebe um objeto JSON com os campos nome, email e cidade_id e atualiza o registro identificado pelo parâmetro de rota id. Todos os campos são obrigatórios.

```
@app.route("/clientes/<int:cliente_id>", methods=["PUT"])
def put_cliente(cliente_id):
    if db.buscar_cliente(cliente_id) is None:
        return jsonify({"erro": "Cliente não encontrado."}), 404

    dados = request.get_json(silent=True) or {}
    nome = dados.get("nome", "").strip()
    email = dados.get("email", "").strip()
    cidade_id = dados.get("cidade_id")

    if not nome or not email or not cidade_id:
        return jsonify({
            "erro": "Os campos 'nome', 'email' e 'cidade_id' são obrigatórios."
        }), 400

    if db.alterar_cliente(cliente_id, nome, email, int(cidade_id)):
        return jsonify({"mensagem": "Cliente atualizado com sucesso."}), 200
    return jsonify({"erro": "E-mail já utilizado por outro cliente."}), 409
```

Corpo da requisição (JSON):

nome *	string – novo nome do cliente
email *	string – novo e-mail (deve ser unico)
cidade_id *	integer – novo ID de cidade

Respostas possíveis:

200 OK	{ "mensagem": "Cliente atualizado com sucesso." }
400 Bad Request	{ "erro": "Os campos nome, email e cidade_id são obrigatórios." }
404 Not Found	{ "erro": "Cliente não encontrado." }
409 Conflict	{ "erro": "E-mail ja utilizado por outro cliente." }

DELETE /clientes/<id> *Remove um cliente pelo ID*

Remove o cliente identificado pelo parâmetro de rota id. Retorna 404 se o ID não existir.

```
@app.route("/clientes/<int:cliente_id>", methods=["DELETE"])
def delete_cliente(cliente_id):
    if db.buscar_cliente(cliente_id) is None:
        return jsonify({"erro": "Cliente nao encontrado."}), 404
    db.excluir_cliente(cliente_id)
    return jsonify({"mensagem": "Cliente removido com sucesso."}), 200
```

Respostas possíveis:

200 OK	{ "mensagem": "Cliente removido com sucesso." }
404 Not Found	{ "erro": "Cliente nao encontrado." }

Passo 6: Código Completo do api.py

Abaixo o código completo e consolidado do módulo api.py:

```
# api.py
from flask import Flask, jsonify, request
import database as db

app = Flask(__name__)
db.criar_tabelas()

# -----
# CIDADES
# -----

@app.route("/cidades", methods=["GET"])
def get_cidades():
    return jsonify(db.listar_cidades()), 200

@app.route("/cidades", methods=["POST"])
def post_cidade():
    dados = request.get_json(silent=True) or {}
    nome = dados.get("nome", "").strip()
    if not nome:
        return jsonify({"erro": "O campo 'nome' e obrigatorio."}), 400
    if db.incluir_cidade(nome):
        return jsonify({"mensagem": f"Cidade '{nome}' cadastrada."}), 201
    return jsonify({"erro": "Ja existe uma cidade com esse nome."}), 409

@app.route("/cidades/<int:cidade_id>", methods=["DELETE"])
def delete_cidade(cidade_id):
    resultado = db.excluir_cidade(cidade_id)
    if resultado is None:
        return jsonify({"erro": "Cidade nao encontrada."}), 404
    if not resultado:
        return jsonify({"erro": "Ha clientes vinculados a esta cidade."}), 409
    return jsonify({"mensagem": "Cidade removida."}), 200

# -----
# CLIENTES
# -----

@app.route("/clientes", methods=["GET"])
def get_clientes():
    return jsonify(db.listar_clientes()), 200

@app.route("/clientes/<int:cliente_id>", methods=["GET"])
def get_cliente(cliente_id):
    cliente = db.buscar_cliente(cliente_id)
    if cliente is None:
        return jsonify({"erro": "Cliente nao encontrado."}), 404
    return jsonify(cliente), 200

@app.route("/clientes", methods=["POST"])
def post_cliente():
    dados = request.get_json(silent=True) or {}
    nome = dados.get("nome", "").strip()
```

```

        email      = dados.get("email", "").strip()
        cidade_id = dados.get("cidade_id")
        if not nome or not email or not cidade_id:
            return jsonify({"erro": "nome, email e cidade_id sao obrigatorios."}),
400
        if db.incluir_cliente(nome, email, int(cidade_id)):
            return jsonify({"mensagem": "Cliente cadastrado."}), 201
        return jsonify({"erro": "E-mail ja cadastrado."}), 409

@app.route("/clientes/<int:cliente_id>", methods=["PUT"])
def put_cliente(cliente_id):
    if db.buscar_cliente(cliente_id) is None:
        return jsonify({"erro": "Cliente nao encontrado."}), 404
    dados      = request.get_json(silent=True) or {}
    nome       = dados.get("nome", "").strip()
    email      = dados.get("email", "").strip()
    cidade_id = dados.get("cidade_id")
    if not nome or not email or not cidade_id:
        return jsonify({"erro": "nome, email e cidade_id sao obrigatorios."}),
400
    if db.alterar_cliente(cliente_id, nome, email, int(cidade_id)):
        return jsonify({"mensagem": "Cliente atualizado."}), 200
    return jsonify({"erro": "E-mail ja utilizado por outro cliente."}), 409

@app.route("/clientes/<int:cliente_id>", methods=["DELETE"])
def delete_cliente(cliente_id):
    if db.buscar_cliente(cliente_id) is None:
        return jsonify({"erro": "Cliente nao encontrado."}), 404
    db.excluir_cliente(cliente_id)
    return jsonify({"mensagem": "Cliente removido."}), 200

# _____
# INICIALIZACAO
# _____

if __name__ == "__main__":
    app.run(debug=True)

```

Passo 7: Executando e Testando a API

7.1 Iniciando o Servidor

Com o ambiente virtual ativado, execute:

```
python api.py
```

O Flask iniciará em modo de desenvolvimento na porta 5000. A saída esperada é:

```
* Running on http://127.0.0.1:5000
* Debug mode: on
```

7.2 Testando com curl

Você pode testar todos os endpoints com o utilitário curl a partir de outro terminal. Abaixo exemplos para cada operação:

Listar cidades:

```
curl http://localhost:5000/cidades
```

Cadastrar cidade:

```
curl -X POST http://localhost:5000/cidades \
-H "Content-Type: application/json" \
-d '{"nome": "Belo Horizonte"}'
```

Cadastrar cliente:

```
curl -X POST http://localhost:5000/clientes \
-H "Content-Type: application/json" \
-d '{"nome": "Ana Lima", "email": "ana@email.com", "cidade_id": 1}'
```

Atualizar cliente (ID 1):

```
curl -X PUT http://localhost:5000/clientes/1 \
-H "Content-Type: application/json" \
-d '{"nome": "Ana P. Lima", "email": "ana.p@email.com", "cidade_id": 1}'
```

Excluir cliente (ID 1):

```
curl -X DELETE http://localhost:5000/clientes/1
```

7.3 Testando com Postman ou Insomnia

Alternativamente, utilize ferramentas gráficas como Postman ou Insomnia. Configure cada requisição da seguinte forma:

- URL base: `http://localhost:5000`
- Header obrigatório para POST e PUT: `Content-Type: application/json`
- Corpo das requisições POST/PUT: raw JSON com os campos especificados em cada endpoint

7.4 Observações Finais

- O modo debug=True reinicia o servidor automaticamente a cada alteração no código.
- Para produção, utilize um servidor WSGI como Gunicorn no lugar do servidor embutido do Flask.
- O banco de dados clientes.db é criado automaticamente na primeira execução.
- A API não implementa autenticação.

--- Fim do Tutorial ---